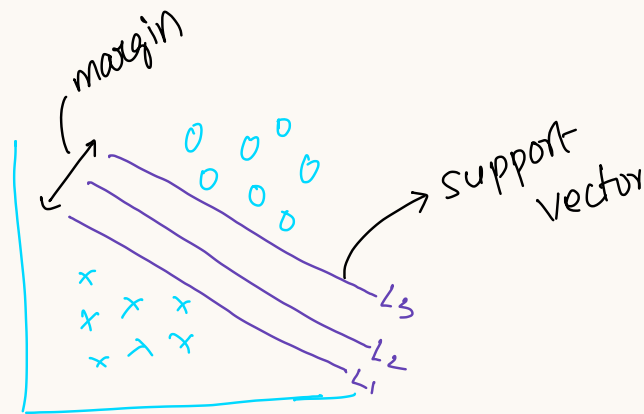


SVM

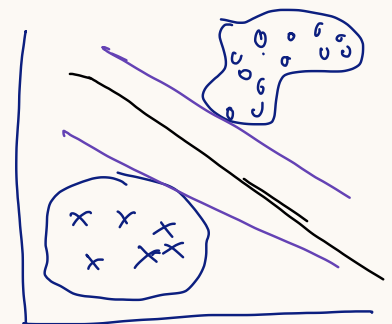


Why?

1. Better prediction confidence
2. Can handle high dimensional data effectively
3. Better with non-linear data
4. Handles imbalanced data and outliers well.

What?

— It builds a boundary in such a way that, minimal distance distance btw the margins.

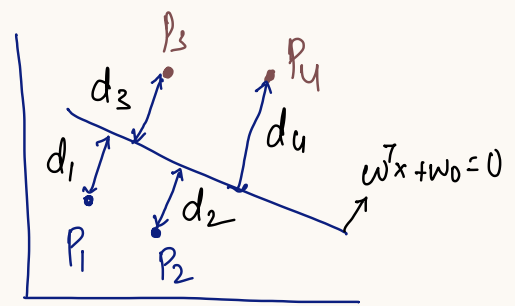


— And it is equi distance for both the margins.

— It assumes that the margins will pass through the nearest points to the boundary

How classification algorithm works?

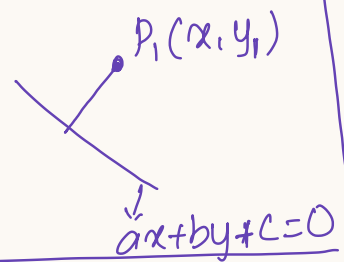
— How algorithms work is draw a $\perp$ line btw point & line and find the distance.



$$d_1 = \frac{(w^T x + w_0) @ P_1}{\sqrt{w_1^2 + w_2^2 + w_3^2 + w_4^2}}$$

Finding the $\perp$ distance btw the point & line is

$$\text{distance } P_1 = \frac{ax_1 + by_1 + c}{\sqrt{a^2 + b^2}}$$



Distance btw plane $(ax + by + cz + d = 0)$ & point (x, y, z) is

$$P = \frac{ax_1 + by_1 + cz_1 + d}{\sqrt{a^2 + b^2 + c^2}}$$

In the same way distance for P_3 point is

$$d_3 = \frac{(w^T x + w_0) @ P_3}{\sqrt{w_1^2 + w_2^2 + \dots + w_n^2}}$$

$$\|w\| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$

$$= \frac{(w^T x + w_0) @ P_3}{\|w\|} \rightarrow \text{normalizing hyperplane}$$

$$\frac{w_1}{\|w\|} x_1 + \frac{w_2}{\|w\|} x_2 + \dots + \frac{w_n}{\|w\|} x_n + w_0 = 0$$

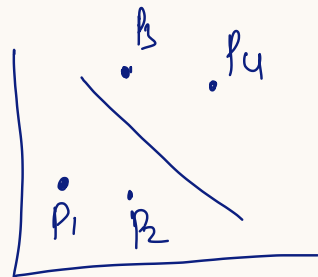
when we normalizing all the coefficients

$$\sqrt{\left(\frac{w_1}{\|w\|}\right)^2 + \left(\frac{w_2}{\|w\|}\right)^2 + \dots + \left(\frac{w_n}{\|w\|}\right)^2} = 1$$

Then $d_3 = \frac{w^T x + w_0 @ \beta_3}{\|w\| \rightarrow 1} \Rightarrow w^T x + w_0$

$\therefore d_3 = \text{dot product btw } (w \cdot x) \text{ where } 'w' \text{ are normalized.}$

If P_3, P_4 are directions of w , then if you find the distance d_3 & d_4 both are greater than 0.



— If P_1, P_2 are opposite direction d_1 & $d_2 < 0$.

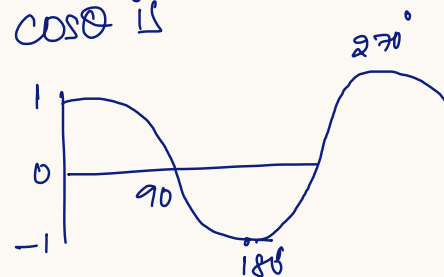
Another way to find the dot product btw w & x is

$$\|w\| \cdot \|x\| \cdot \cos \theta$$

$\Rightarrow$ if the angle θ btw $0-90^\circ \rightarrow +ve$

if θ is $90-180^\circ \rightarrow -ve$.

$\cos \theta$ is

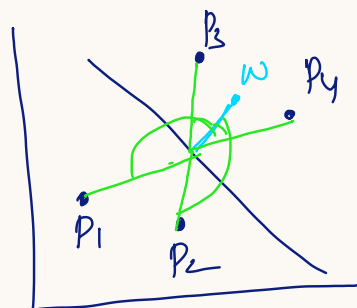


— In order to classify points

Find out dot product btw

coeff of plane $(w) \cdot \text{point}(x) > 0 \rightarrow +ve$

$w \cdot x < 0 \rightarrow -ve$



How point predicted correct or not?

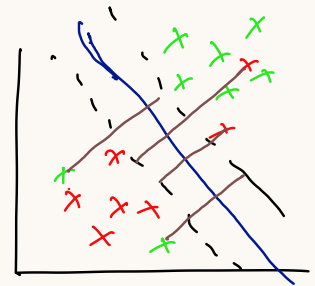
Loss function for Hard margin is:

$$\operatorname{argmax}_{(w^*, b^*)} = \frac{2}{\|w\|}$$

- But the problem with Hard margin is, it will not work for miss classification (non-linear)

Loss function for Soft margin is:

$$\operatorname{argmin}_{(w^*, b^*)} = \frac{\|w\|}{2} + C \sum_{i=1}^n \xi_i$$



- In Hard margin we are trying to maximize the margin distance.

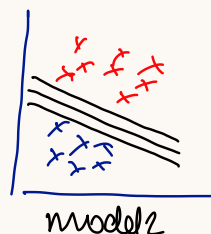
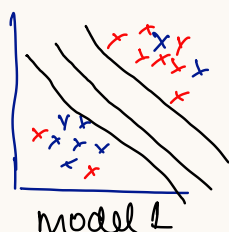
- But in Softmargin we are trying to minimize error

$$\operatorname{argmin} = \text{margin error} + \text{classification error}$$

$\sum \xi_i \rightarrow$ is sum of distance of all misclassified points

$\sum \xi_i = 0 \rightarrow$ if all points are correctly classified.

$C \rightarrow$ is an hyper parameter



- model 1 - margin distance high but miss classified
- model 2 margin distance low but correctly classified

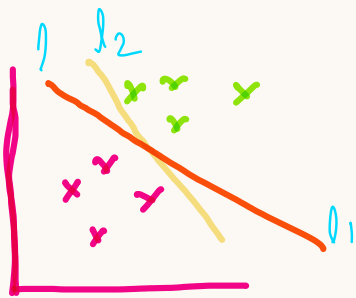
Which is better one, this where 'C' will tune.

'C' $\rightarrow$ should be small values like (0.001, 0.001...)

'C' default is 1.

- If you want to increase margin distance, $C \uparrow$
set the 'C' value to high

- If we want to reduce classification error ξ_i ,
decrease 'C' value ($C \times \xi_i \rightarrow (0.001 \times \text{anything}) \Rightarrow \downarrow$)

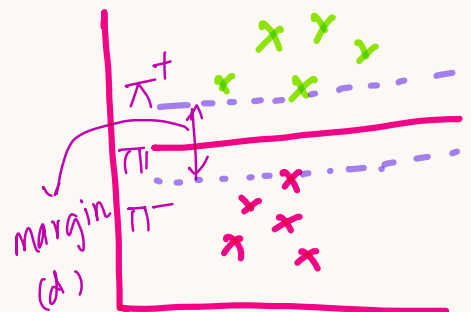


- Even both l_1 & l_2 classifies the data points well. Still ' l_1 ' is most best line which maximize the distance and will classify better for future data points.

- Distance b/w the point & plane is margin.

- we should find a plane/line which maximize margin.

- First Find the best fit line



and move the line parallelly to upward/forward
direct reach/touch the 1st data point.

— Do the same for backward and touch the
first point. Those 2 points are your support
vectors.

- SVM is robust with outlier
- work well with non-linear data.
- work for both classification & Regression.

Now How to find the distance of d ?

Let's try to create one Decision Rule.

1. Draw a line from origin to

line (w)

2. New data point came for prediction. Find $\vec{u}$

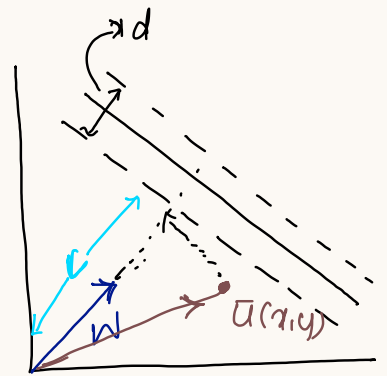
3. Do projection the point $(\vec{u})$ on $\vec{w}$

As we don't know the distance of $\vec{w}$. Let's
take some value C .

$$\therefore \vec{w} \cdot \vec{u} \geq C.$$

$$\vec{w} \cdot \vec{u} - C \geq 0. \quad [\text{For generalizing replace } C \text{ with } b]$$

$$\boxed{\vec{w} \cdot \vec{u} + b \geq 0}$$



- We know (w, b) from the best fit line

- From the above equation, the value is

+ve $\rightarrow$ +ve value

-ve $\rightarrow$ -ve value.

For Ex: line equ is $2x + 3y + 3 = 0$.

means $w_1x_1 + w_2x_2 + b = 0$.

$w_1 = 2, w_2 = 3, b = 3$.

new point is $(5, 2)$

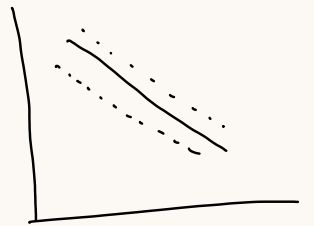
$$\Rightarrow 2(5) + 3(2) + 3 \geq 0$$

+ve value $\rightarrow$ +ve class.

same for -ve value for -ve class.

$\therefore$ For any x_i

$$\hat{y}_i = \begin{cases} +1 & \text{if } \vec{w} \cdot \vec{x} + b \geq 0 \\ -1 & \text{if } \vec{w} \cdot \vec{x} + b \leq 0. \end{cases}$$



Now we need to find the equation for hyperplane.

- we need to find a plane equ with best (w, b) values which maximize the distance (margin) 'd'.

— Now we are assuming some line equations for our marginal lines.

$$\begin{array}{l} \text{---} w^T x + b = 1 \\ \text{---} w^T x + b = 0 \\ \text{---} w^T x + b = -1 \end{array}$$

By using these equ. we are trying to find 'd'

Note 1: The distance d_1 & d_2 should be equal

Note 2: Why we are considering (1 & -1). ~~The~~ no particular reason for simplicity we are taking (1 & -1) but can be any ($2r, -2r$)

— Now we need to find an optimization, where the marginal distance (d) is max and no point should be inside the lines.

$$\therefore \vec{w} \cdot \vec{x} + b \geq 1 \rightarrow \text{for +ve class}$$

$$\vec{w} \cdot \vec{x} + b \leq -1 \rightarrow \text{for -ve class.}$$

[$\therefore$ in the above equ = is for support vectors]

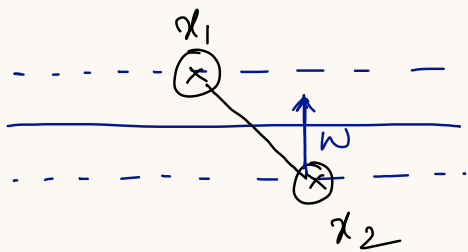
But For better simplification let's try to

convert 2 eqn into 1.

$$\left. \begin{array}{l} \vec{w} \cdot \vec{x} + b \geq 1 \\ \vec{w} \cdot \vec{x} + b \leq -1 \end{array} \right\} \Rightarrow \begin{array}{l} y_i (\vec{w} \cdot \vec{x} + b) \geq 1 \\ y_i (\vec{w} \cdot \vec{x} + b) \leq -1 \end{array}$$

$\therefore$ Final eqn is $y_i (\vec{w} \cdot \vec{x} + b) \geq 1$

For support vector $y_i (\vec{w} \cdot \vec{x} + b) = 1$



Find the distance b/w the support vectors (x_1 & x_2)

$(x_2 - x_1)$ Find the shortest distance.

- Draw a unit vector(w) $\Rightarrow \frac{w}{\|w\|}$

$$\begin{aligned} \therefore d &= (\vec{x}_2 - \vec{x}_1) \cdot \frac{\vec{w}}{\|\vec{w}\|} \\ &= \frac{x_2 \cdot \vec{w} - x_1 \cdot \vec{w}}{\|\vec{w}\|} \end{aligned}$$

From above eqn $y_i (\vec{w} \cdot \vec{x}_i + b) = 1$.

if +ve $\rightarrow 1 (\vec{w} \cdot \vec{x}_2 + b) = 1$.

$$\vec{w} \cdot \vec{x}_2 = 1 - b$$

if -ve $\rightarrow -1 (\vec{w} \cdot \vec{x}_1 + b) = 1$.

$$w \bar{x}_1 = -b - 1$$

$$\therefore \frac{(1-b) - (-b-1)}{\|w\|}$$

$$= \frac{1 - \cancel{b} + \cancel{b} + 1}{\|w\|} = \frac{2}{\|w\|} \Rightarrow \text{distance } d:$$

$$\therefore \operatorname{argmax}_{(m,b)} \frac{2}{\|w\|} \text{ such that } y_i(\bar{w} \cdot \bar{x} + b) \geq 1$$

- Basically this equation will not handle misclassified points.

- This equation works well, when the data is linearly separable. We ~~are~~ calling it a

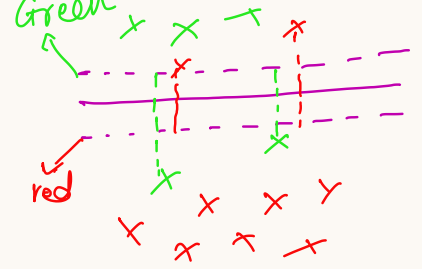
Hard Margin:

- That is why we are doing some changes in the above equ to handle miss classified point.

$$\max f(x) \approx \min \frac{1}{f(x)}$$

$$\therefore \operatorname{argmin}_{(m^*, b^*)} \frac{\|w\|}{2} \text{ is the loss function}$$

with soft margin-



→ For handling miss classified points,

→ Find the distance of each miss class from its marginal line (ξ_i)

→ Along with that multiply with a hyper parameter

(C)

Final equ is

$$\argmin_{(w, b)} \frac{\|w\|}{2} + C \sum \xi_i$$

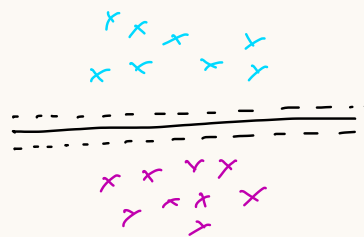
$\sum \xi_i$ - sum of distances

C - hyper parameter.

$\therefore$ $\downarrow$
marginal error + classification error

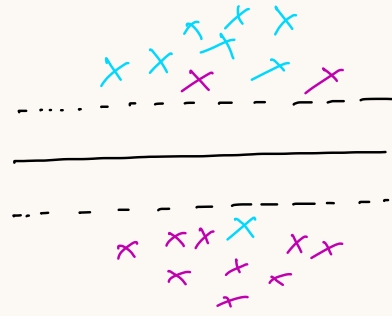
Note: 'C' can be any value.

= If 'C' value is high classification error will be high, means ignoring marginal distance.



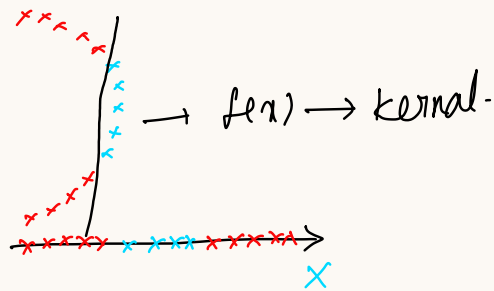
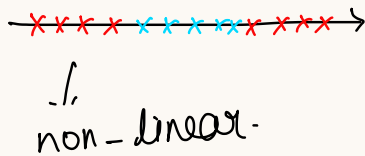
→ Basically marginal distance is low but classifying well

— If 'c' value is very low like 0.01, 0.001...
marginal distance is high, but can have
miss classification.



Kernel Trick :

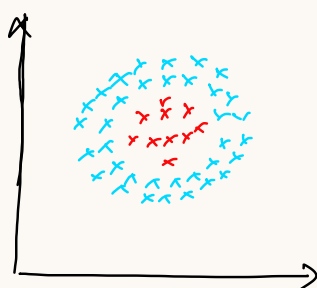
1D



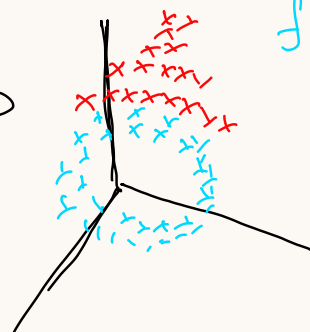
- kernel trick is used for handling non-linear data
- kernel trick is a kind of function which helps us to bring the data into higher dimensions.

Ex: $1D \rightarrow 2D$.

Types $\rightarrow$ RBF
polynomial
sigmoid



$2D \rightarrow 3D$



$$f(x) = e^{-x^2}$$

Main idea is using kernel function perform
kernel transformation to higher dimension which
will make ~~it~~ the data linearly separable.

Core idea of SVM:

- SVM does not just find a separating line.
- It finds a line with the maximum margin.
- Nearest points are called support vectors.

Decision boundary is $w^T x + b = 0$.

Two margin line are

$$w^T x + b = 1$$

$$w^T x + b = -1.$$

Distance btw these lines are

$$\text{Margin} = \frac{2}{\|w\|}$$

- If weights are large $\rightarrow$ margin small
- If weights are small $\rightarrow$ margin large.

Optimization equ: $\min \frac{1}{2} \|w\|^2$

where $y_i (w^T x_i + b) \geq 1$.

↓
This ensure points stay outside margin
simply linearly separable.

For non-linear data

updated constraint is

$$y_i(w^T x + b) \geq 1 - \xi_i$$

Loss function becomes

$$\frac{1}{2} \|w\|^2 + C \sum \xi_i$$

where 'c' is penalty parameter.

Large 'c' value $\rightarrow$ Fewer mistakes

Small 'c' value $\rightarrow$ larger margin

— In SVM, only a few points determine a boundary.
(support vectors)

All other points are ignored.

This makes SVM memory efficient.

— In SVR, instead of margin between the classes,
SVR uses, points inside the tube

— Loss only applied if predictions goes outside tube.

$$\text{Loss function of SVR} = \frac{1}{2} \|w\|^2$$

$$\text{where } |y_i - f(x)| \geq \varepsilon.$$

Advantages of SVM:

1. Works well in high dimensional data
2. Effective in text classification
3. Robust to Outliers.
4. works with small datasets.

Why SVM robust to outliers?

1. It focuses only on the most imp points (support vector)
 - unlike many algorithms that uses all data points equally, svm only cares about the points closest to decision boundary.
 - [points far away from the boundary do not affect the model].
 - This prevents model to memorize noise.

2. SVM chooses the boundary that maximize margin

small margin = model sensitive to small noise

large margin = model more stable.

— SVM chooses the safest boundary, not just a correct boundary.

3. Regularization Parameter (C).

' C ' that controls overfitting.

Large ' C ' = tries to classify each point correctly
(over fitting)

small ' C ' = allows some errors = better generalization

4. Kernel trick controls complexity

2D data $\rightarrow$ mapped to higher dimension $\rightarrow$

becomes linearly separable.

"SVM avoids overfitting because it learns only from the most difficult points and chooses the boundary with the largest safety margin".

